

- Description

This module manages a smart vending machine system, ensuring that:

1. The machine continuously monitors its inventory level to ensure sufficient stock.
2. When a purchase is requested, it checks if the requested quantity is available.
3. The total cost is calculated based on the unit price if the purchase is successful.
4. If the current inventory falls below 20% of the total inventory, a refill warning is triggered.

This system helps maintain adequate inventory levels, prevents overselling, and notifies operators when a refill is needed.

- Specification

module SmartVendingMachineInventory

var currentInventory, totalInventory: integer

var pricePerItem, totalCost: real

process purchaseItems(quantity: integer) returns (totalCost: real)

functional scenario form:

quantity <= currentInventory && totalCost = quantity * pricePerItem,
currentInventory = currentInventory – quantity, Purchase successful, totalCost =
quantity * pricePerItem, currentInventory updated.

||

quantity > currentInventory && totalCost = 0, Purchase failed, totalCost = 0,
currentInventory unchanged.

end process

end module

- Java Code

```
package code;
class SmartVendingMachine {
    private int totalInventory;
    private int currentInventory;
    private double pricePerItem;
    public SmartVendingMachine(int totalInventory, double pricePerItem)
    {
        this.totalInventory = totalInventory;
        this.currentInventory = totalInventory;
        this.pricePerItem = pricePerItem;
    }
    public void purchaseItems(int quantity) {
        if (quantity <= 0) {
            System.out.println("Invalid quantity requested.");
            return;
        }
    }
}
```

```

        if (quantity > currentInventory) {
            System.out.println("Purchase failed: Not enough inventory."
);
        } else {
            currentInventory -= quantity;
            double totalCost = quantity * pricePerItem;
            System.out.printf("Purchase successful. Total cost: $%.2f.
Remaining inventory: %d\n", totalCost, currentInventory);
        }
        if (currentInventory < totalInventory * 0.2) {
            System.out.println("Warning: Inventory below 20%. Please re
fill.");
        }
    }

    public int getCurrentInventory() {
        return currentInventory;
    }
}

public class c70_SmartVendingMachineSystem {
    public static void main(String[] args) {
        SmartVendingMachine machine = new SmartVendingMachine(100, 2.50
);

        // testcase-VT:
        // machine.purchaseItems(30);
        // machine.purchaseItems(50);
        // machine.purchaseItems(25);
        // machine.purchaseItems(10);
        // machine.purchaseItems(-10);

        // testcase-FT:
        // machine.purchaseItems(222);
        // machine.purchaseItems(36);
        // machine.purchaseItems(548);
        // machine.purchaseItems(35964100);
        // machine.purchaseItems(74543473);

        // testcase-Z3:
        // machine.purchaseItems(30);
        // machine.purchaseItems(50);
        // machine.purchaseItems(25);
        // machine.purchaseItems(10);
        // machine.purchaseItems(-10);

```

```

// testcase-UVT:
machine.purchaseItems(30);
machine.purchaseItems(50);
machine.purchaseItems(25);
machine.purchaseItems(10);
machine.purchaseItems(-10);
}
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c70_SmartVendingMachineSystem.java	100.0 %	6	0	6
> c70_SmartVendingMachineSystem		0	0	0
> SmartVendingMachine	100.0 %	6	0	6

3. Vibration testing instruction coverage: 93.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.4 %	91	20,805	20,896
✓ src	0.4 %	91	20,805	20,896
✓ code	0.4 %	91	20,805	20,896
✓ c70_SmartVendingMachineSystem.java	93.8 %	91	6	97
> c70_SmartVendingMachineSystem	88.0 %	22	3	25
> SmartVendingMachine	95.8 %	69	3	72

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	838	842
✓ src	0.5 %	4	838	842
✓ code	0.5 %	4	838	842
✓ c70_SmartVendingMachineSystem.java	66.7 %	4	2	6
> c70_SmartVendingMachineSystem		0	0	0
> SmartVendingMachine	66.7 %	4	2	6

5. Fuzz testing instruction coverage: 86.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.4 %	84	20,812	20,896
✓ src	0.4 %	84	20,812	20,896
✓ code	0.4 %	84	20,812	20,896
✓ c70_SmartVendingMachineSystem.java	86.6 %	84	13	97
> c70_SmartVendingMachineSystem	88.0 %	22	3	25
> SmartVendingMachine	86.1 %	62	10	72

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c70_SmartVendingMachineSystem.java	100.0 %	6	0	6
> c70_SmartVendingMachineSystem		0	0	0
> SmartVendingMachine	100.0 %	6	0	6

7. Z3 instruction coverage: 93.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	91	20,805	20,896
▼ src	0.4 %	91	20,805	20,896
▼ code	0.4 %	91	20,805	20,896
▼ c70_SmartVendingMachineSystem.java	93.8 %	91	6	97
> c70_SmartVendingMachineSystem	88.0 %	22	3	25
> SmartVendingMachine	95.8 %	69	3	72

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c70_SmartVendingMachineSystem.java	100.0 %	6	0	6
> c70_SmartVendingMachineSystem		0	0	0
> SmartVendingMachine	100.0 %	6	0	6

9. Unimproved V-method instruction coverage: 93.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	91	20,805	20,896
▼ src	0.4 %	91	20,805	20,896
▼ code	0.4 %	91	20,805	20,896
▼ c70_SmartVendingMachineSystem.java	93.8 %	91	6	97
> c70_SmartVendingMachineSystem	88.0 %	22	3	25
> SmartVendingMachine	95.8 %	69	3	72